A Floating-Point SRAM Computing-in-Memory Macro Using Digital-Domain Structure for CNNs

Wenjuan Lu, *Member, IEEE*, Can Zhang, Wei Hu, Xiaobo Gong, Zhihao Li, Shijie Xu, Xiulong Wu, *Senior Member, IEEE*, and Chunyu Peng, *Senior Member, IEEE*

***Abstract*—With the increasingly complex computing requirements of convolutional neural networks (CNNs) in edge computing scenarios, computing-in-memory (CIM) has been proposed to address the bottlenecks of high data transfer energy consumption and long computing latency faced by traditional von Neumann architecture. This work proposes a floating-point (FP) CIM macro based on static random access memory (SRAM), which achieves high-efficiency CNN matrix multiplication and accumulation (MAC) operations. The main features of this work include: 1) dynamically configuring the maximum exponent to eliminate the exponent comparison operation while maintaining high inference accuracy; 2) introducing a dual-bit wordline-separated 6T SRAM cell (DWScell) and a mantissa computing cell (MCC) for full-precision and high-throughput mantissa multiplication; and 3) proposing a tailored 9T full adder (FA) and a 5T half adder (HA) to reduce the delay and area cost of both exponent and mantissa calculation. In a 28-nm CMOS process simulation environment, this macro achieved an energy efficiency (EF) of 4.81 TFLOPS/W when performing MAC operations on BF16-input and BF16-weight. Based on the ResNet50 network model, classification verification of the CIFAR-100 dataset achieved an accuracy of 76.7%.**

***Index Terms*—Computing-in-memory (CIM), convolutional neural networks (CNNs), dynamic configuration, floating-point (FP), multiplication and accumulation (MAC).**

I. Introduction

FROM image processing to computer vision, convolutional neural networks (CNNs) have demonstrated increasingly broad application prospects. However, the exponential expansion of data volume has exacerbated the "memory wall" and "power wall" challenges in traditional von Neumann architecture-based chips [1], [2], [3], [4]. The computing-in-memory (CIM) architecture addresses these challenges by integrating memory and processing units, significantly reducing data transmission latency and energy consumption [5].

Early research on SRAM-based CIM primarily focused on integer (INT) computation [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], which often incurred significant precision loss in complex neural network training and inference models. In contrast, floating-point (FP) arithmetic offers superior precision, making it more suitable for such applications [16], [17]. While several studies have explored analog-domain CIM architectures [18], [19], [20], [21], [22], [23], the inherent nonlinearity of MOS transistors necessitates significantly larger area overhead and higher power consumption to achieve both high precision and reliability. In contrast, digital-domain CIM [24], [25], [26], [27], [28] leverages digital circuit techniques, circumventing the noise sensitivity and error accumulation issues associated with analog implementations. As a result, they achieve higher computational precision and exhibit greater robustness against temperature fluctuations and process variations.

Many studies on digital-domain FP-CIM have been proposed. Mai et al. [29] developed a configurable SRAM-based CIM macro that supports flexible precision switching between BF16 and FP32 formats through accumulator reconfiguration, thereby enabling high-precision FP MAC operations. However, exponent comparison along with redundant mantissa alignment operations results in suboptimal energy efficiency. Saikia et al. [30] proposed a CIM macro that supports both FP8 and BF8 formats. In FP8 mode, the macro performs standard FP MAC operations, while in the BF8 mode, the use of a shared exponent within the same tensor group significantly simplifies the computation process. However, this shared-exponent design is merely appropriate for those scenarios where the data distribution is highly concentrated. Guo et al. [31] proposed an FP-CIM macro that uses double-bit cells and dedicated computing units to construct a mantissa array. To reduce internal bandwidth and area cost, the design uses full-precision computation on the higher bits while applying approximation to the lower bits. However, this approach introduces deterministic computational errors, necessitating additional compensation through LUT-based training.

In summary, existing FP-CIM architectures often struggle to achieve an optimal balance among energy efficiency, throughput, and numerical precision. These limitations stem primarily from three sources: 1) the substantial area and energy overhead of the exponent processing circuit, which performs summation, comparison, and subtraction; 2) the redundant alignment overhead incurred by excessive mantissa shifting; and 3) the inherent tradeoff between computational accuracy and hardware cost in mantissa multiplication. To address

Received 17 September 2025; revised 11 December 2025 and 12 January 2026; accepted 17 January 2026. Date of publication 6 February 2026; date of current version 28 April 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62471003; in part by the University Synergy Innovation Program of Anhui Province under Grant GXXT-2023-003, Grant GXXT-2023-011, and Grant GXXT-2023-012; and in part by the National Natural Science Foundation of the Higher Education Institutions of Anhui Province under Grant 2023AH040011. *(Corresponding author: Xiulong Wu.)*

The authors are with the School of Integrated Circuits, Anhui University, Hefei 230601, China (e-mail: xiulong@ahu.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2026.3656424

1063-8210 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

these challenges, we present a fully digital SRAM-based FP-CIM macro that incorporates three key innovations: 1) a dynamic maximum exponent configuration scheme based on long-tail distribution that removes the exponent comparator, reuses the adder/subtractor circuit, and eliminates unnecessary mantissa alignment; 2) a mantissa calculation array comprising DWScell and mantissa computing cell (MCC) units which enables full-precision and high-throughput multiplication; and 3) customized 9T full adder (FA) and 5T half adder (HA) designs that provide area-efficient and error-free operation. This macro achieves improvements in both energy and area efficiency while maintaining inference accuracy.

The remainder of this article is organized as follows. Section II introduces the overall structure of the proposed FP-CIM macro. Section III presents the exponent computation scheme. Section IV presents the mantissa computation scheme. Section V describes the simulation results and performance analysis. Section VI summarizes this work.

II. Digital-Domain FP-CIM Structure

A. FP Calculation Rules

CNN training and inference processes involve extensive matrix multiplication operations. When using FP arithmetic, the fundamental computational operator becomes the FP multiplication and accumulate (FPMAC). Consider four FP operands: input activations IN0 and IN1, and weights $W0$ and $W1$, where $\mathrm{IN0} = 2^{E_{\mathrm{IN0}}} M_{\mathrm{IN0}}$, $\mathrm{IN1} = 2^{E_{\mathrm{IN1}}} M_{\mathrm{IN1}}$, $W0 = 2^{E_{W0}} M_{W0}$, and $W1 = 2^{E_{W1}} M_{W1}$.

The FPMAC operation proceeds through the following computational stages:

$$S = \mathrm{IN0} \times W0 + \mathrm{IN1} \times W1. \tag{1}$$

And we can expand the formula

$$S = 2^{E_{\mathrm{IN0}}} M_{\mathrm{IN0}} \times 2^{E_{W0}} M_{W0} + 2^{E_{\mathrm{IN1}}} M_{\mathrm{IN1}} \times 2^{E_{W1}} M_{W1} \tag{2}$$

$$= 2^{E_{\mathrm{IN0}}+E_{W0}} M_{\mathrm{IN0}} M_{W0} + 2^{E_{\mathrm{IN1}}+E_{W1}} M_{\mathrm{IN1}} M_{W1} \tag{3}$$

$$= 2^{E_{\max}} (M_0 \gg r_0 + M_1 \gg r_1) \tag{4}$$

where Emax = max $(E_{\mathrm{IN0}} + E_{W0}, E_{\mathrm{IN1}} + E_{W1})$, and r_0 and r_1 are the mantissa shift amount. This formulation indicates that for FPMAC operations, the computational process involves two key phases: initial exponent processing where the maximum exponent value (Emax) is determined from the summed exponents; and subsequent mantissa alignment, where each mantissa is right-shifted by $\Delta R = E_{\max} - E_{\mathrm{sum}}$ bits to ensure proper alignment before final accumulation. This approach maintains numerical precision while optimizing the computation flow.

B. Dynamic Configuration of Emax

Based on the methodology presented in [32], which separates FPMAC into dense and sparse parts according to the long-tail distribution of activation/weight exponents, we further analyze the distribution of Esum during CNN inference. Through validation on widely used models such as VGG and ResNet, as well as public datasets such as CIFAR-10 and ImageNet, we found that Esum similarly exhibits a long-tail distribution, characterized by a concentrated main range and occasional extreme outliers.

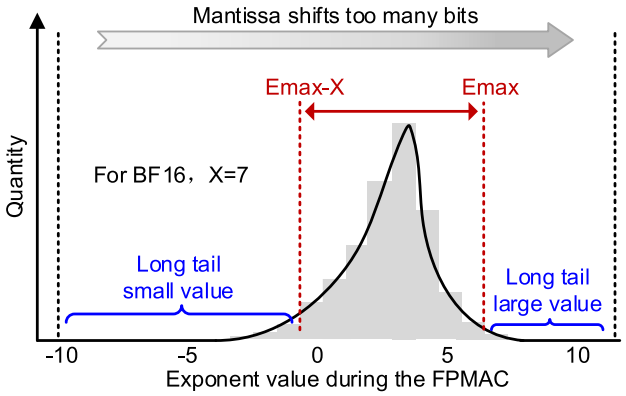


Fig. 1. Long-tail distribution of the exponent and the reasonable allocation of Emax.

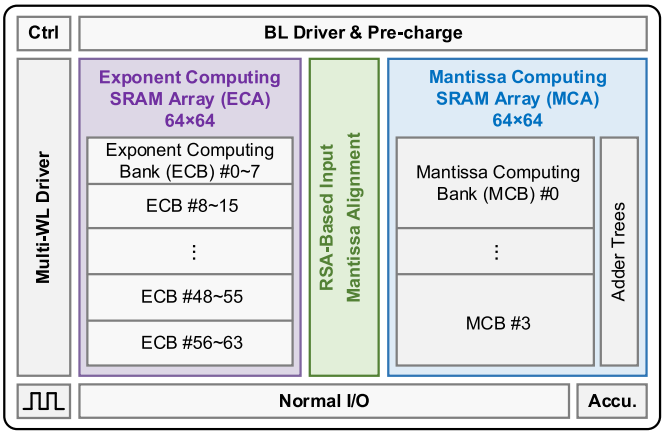


Fig. 2. Structure of the proposed FP-CIM macro.

When aligning mantissas with the true maximum exponent (Emax_true), mantissas associated with smaller exponents require excessive right-shifting operations, leading to substantial loss of significant bits. For BF16 data, a shift amount exceeding 7 bits results in complete loss of meaningful data, thus imposing an upper bound of 7-bit shift operations. Motivated by this observation, we propose a dynamic configuration scheme that adaptively selects an optimal Emax based on exponent value distribution, rather than strictly adopting Emax_true. As illustrated in Fig. 1, the proposed method determines the optimal Emax by constraining the exponent range to [Emax-7, Emax], while maximizing the coverage of exponent values within this bounded interval. This mechanism is implemented off-chip.

C. Proposed CIM Structure

Fig. 2 presents the proposed SRAM-based FP-CIM macro, which incorporates the optimization schemes described in Section I. The macro comprises these key components: a 4-kb exponent computing static random access memory (SRAM) array (ECA), a right-shift-amount (RSA)-based input mantissa alignment (RSAB-IMA) unit, a 4-kb mantissa computing SRAM array (MCA), and a time-scale accumulator. The ECA includes 64 exponent computing banks (ECBs) and the MCA includes four mantissa computing banks (MCBs).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

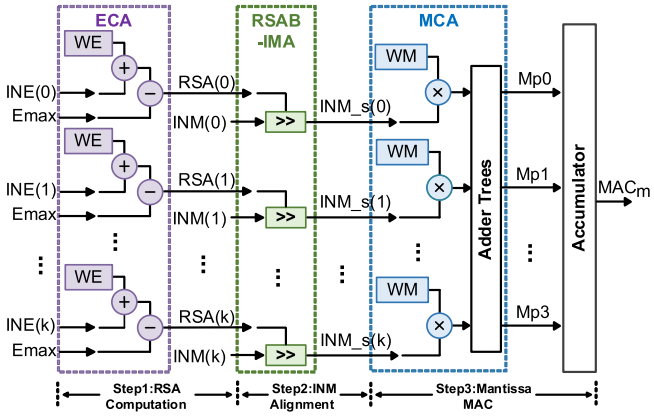


Fig. 3. Calculation process of the proposed FP-CIM.

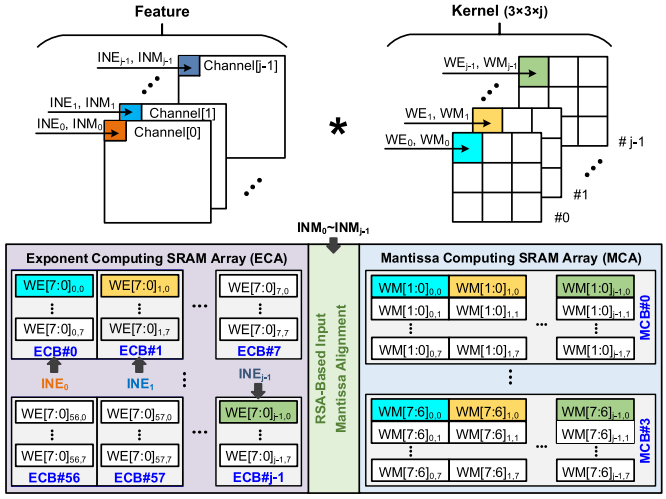


Fig. 4. Proposed 8-kb CIM macro mapping with the convolution layer.

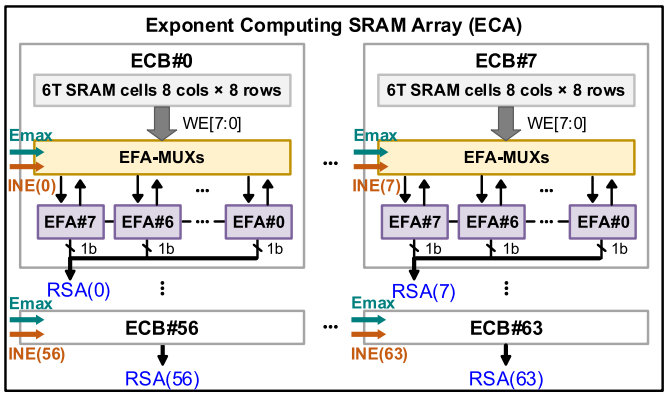


Fig. 5. Structure of the proposed ECA.

The ECA stores weight exponents and computes the arithmetic difference between the configured maximum exponent (Emax) and the summed exponents (Esum), which directly determines the RSA for input mantissa alignment. The RSAB-IMA performs parallel alignment of input mantissa. The MCA cooperates with the accumulator to execute high-precision mantissa multiplication and accumulation (MAC) operations on aligned mantissas. Using an optimized reference exponent (Emax) selection strategy, this macro eliminates the need for conventional maximum exponent detection circuitry while maintaining neural network inference accuracy within 0.1% (@CIFAR10) of FP baseline implementations.

Fig. 3 illustrates the workflow of the proposed CIM macro. For an FPMAC operation with 64 BF16-weights and 64 BF16-inputs, the execution comprises three phases.

1) Exponent processing and shift amount calculation. The ECA first computes the summation of input exponents INE(n) (n = 0–63) and weight exponents WE(n) (n = 0–63) to generate 64 exponent sums [Esum(n)]. Subsequently, it derives the right-shift amounts [RSA(n)] for mantissa alignment by computing RSA = Emax − Esum.
2) Parallel mantissa alignment. RSAB-IMA performs right-shift operations on all 64 input mantissas INM(n) in parallel, using their respective RSA values. This parallel alignment process generates 64 shifted mantissas (INM_s).
3) Mantissa MAC. The MCA performs 64 parallel multiplications between the INM_s and weight mantissa (WM), and then sums the results using four adder trees to yield four Mp values. These values are then supplied to the accumulator to yield the final mantissa result (MACm) of the MAC operation.

D. Convolution Mapping Scheme

The mapping scheme for the CNN of this macro is shown in Fig. 4. Under the IEEE 754 standard, BF16 data are partitioned into a 1-bit sign bit, 8-bit exponent bits, 7-bit mantissa bits, and a 1-bit hidden bit. The weight storage adopts a partitioned organization: The 8-bit weight exponent (WE[7:0]) is stored in the ECA, while the hidden bit and 7-bit WM are converted into an 8-bit mantissa (WM[7:0]) in 2's complement form and stored in the MCA. Each WE occupies eight 6T SRAM cells within the same row of an ECB. For example, ECB#0 stores $WE[7:0]_{0,n}$(n = 0–7), while ECB#1 stores $WE\ [7:0]_{1,n}$(n = 0–7). Each WM is partitioned into four 2-bit segments stored sequentially across four MCBs. Specifically, an 8-bit WM[7:0] is divided into WM[7:6], WM[5:4], WM[3:2], and WM[1:0], stored in MCB#3 ~ 0, respectively. Each kernel#j's weight exponent maps to ECB#j, with its mantissa mapping to the j-th column of MCBs. This partitioned scheme facilitates parallel access to exponent and mantissa components during computation.

III. Exponent Calculation Scheme

A. Overview of ECA

Fig. 5 illustrates the structure of the proposed ECA, which comprises 64 identical ECBs. Each ECB integrates three components: a 6T SRAM array for WE storage, exponent full-adder multiplexers (EFA-MUXs) for operand selection, and eight exponent FAs (EFAs) for arithmetic operations. This design enables parallel exponent processing with optimizing power efficiency.

The ECA computes the shift amounts for input mantissa alignment. Each ECB calculates RSA(n) for INE(n) based on

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

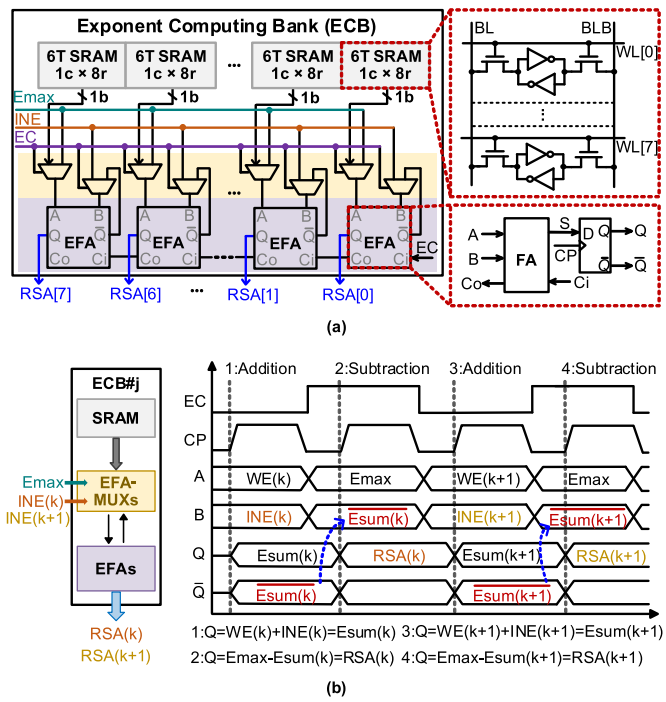


Fig. 6. (a) Structure of ECB. (b) Timing waveform of ECB operation.

the Emax through two stages. In the first stage, the ECB is set to addition mode, and the EFAs sum the 8-bit input exponent INE and the 8-bit weight exponent WE to yield Esum. In the second stage, the ECB is set to subtraction mode, and the EFAs subtract Esum from the Emax to obtain the RSA.

B. Exponent Calculation Details

Fig. 6(a) illustrates the specific structure of the ECB. The SRAM cells connect to eight EFA-MUXs through the bitline (BL). During calculation, only one of the eight horizontal control signals (WL[7:0]) is activated. The EFA-MUXs dynamically route operands to the EFAs according to the current computation phase. Each EFA that comprises a 1b-FA and a D flip-flop implements a 1-bit arithmetic unit. Eight such EFAs are cascaded to form an 8-bit exponent processing unit.

Fig. 6(b) illustrates the timing waveform of the ECB operations. The computation begins by asserting the MUX control signal EC low, configuring the MCB into addition mode. In this mode, the MUX routes the weight exponent WE(k) and input exponent INE(k) to the EFAs' input ports and the EFAs generate the Esum(k): WE(k) + INE(k). The computation result is then latched on the rising edge of the clock signal (CP), with the D flip-flops capturing and outputting both Esum (Q output) and $\overline{E\text{sum}}$ ($\overline{Q}$ output).

Subsequently, the control signal EC is asserted high to configure the MUX for selecting the Emax and Esum as inputs to the EFAs. In this subtraction phase, the carry-in signal (Ci) of EFA#0 is set to logic high to implement 2's complement arithmetic. The EFAs compute the difference (RSA = Emax − Esum), where the output Q is the required RSA. This computational process repeats iteratively for subsequent input exponent INE($k+1$).

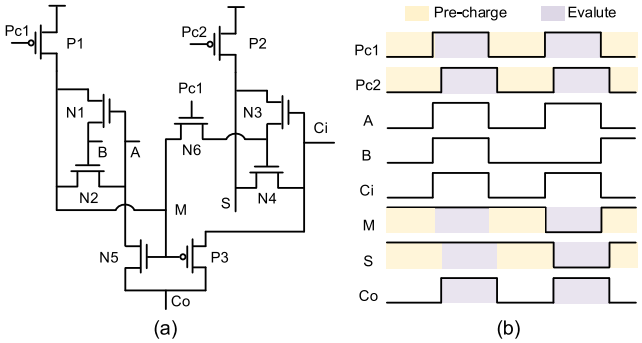


Fig. 7. (a) Proposed 9T FA and (b) its operational waveform.

The 1-bit FA in the EFA uses a novel 9-transistor (9T) design, as shown in Fig. 7(a). The Boolean logic expression for this FA is given by

$$S = A \odot B \odot C$$
$$\text{Co} = (A \odot B)A + \overline{(A \odot B)}\text{Ci}.$$

The 9T FA operates in two computation phases: the precharge phase and the evaluation phase, as shown in Fig. 7(b). During the precharge phase, Pc1 and Pc2 are set to low level, N6 is turned off, and both M and S are precharged to high level. In the evaluation phase, Pc1 and Pc2 are set to high level, N6 is turned on, and computation is performed based on the input signals. It should be noted that Pc2 switches to high level slightly later than Pc1, because the correct calculation of M must be completed before S and Co can be computed. What is more, to achieve full swing output, all the 9T FA output terminals are equipped with buffers.

To place the ECB in conventional memory mode, the control signal EC is asserted high and Emax is held low, thereby disabling the computing unit.

IV. Mantissa Calculation Scheme

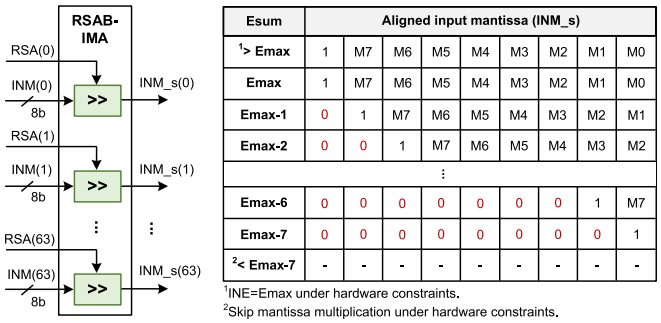
Mantissa computation comprises mantissa alignment and mantissa MAC. To achieve this process, this work proposes the RSAB-IMA for mantissa alignment, the MCA using DWScell for efficient multiplication, and the accumulator for accumulation.

A. Proposed RSAB-IMA Scheme

There are two common methods for parallel alignment of mantissa: one is aligning the product of mantissa multiplication, while the other is aligning the input mantissa. The former method achieves higher computational precision because it performs calculation before shifting, whereas the latter method improves data sparsity by right-shifting input mantissas to generate zero-valued bits, thereby reducing the power consumption. This work adopts the second alignment method and proposes the RSAB-IMA scheme, which is based on RSA to perform parallel shift alignment on the input mantissas.

Fig. 8 shows the operation of the RSAB-IMA scheme. Based on RSAs obtained from ECAs, 64 shift units simultaneously shift the input mantissas so that the product of the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.



Esum	Aligned input mantissa (INM_s)								
[1]> Emax	1	M7	M6	M5	M4	M3	M2	M1	M0
Emax	1	M7	M6	M5	M4	M3	M2	M1	M0
Emax-1	0	1	M7	M6	M5	M4	M3	M2	M1
Emax-2	0	0	1	M7	M6	M5	M4	M3	M2
⋮									
Emax-6	0	0	0	0	0	0	0	1	M7
Emax-7	0	0	0	0	0	0	0	0	1
[2]< Emax-7	-	-	-	-	-	-	-	-	-

[1]INE=Emax under hardware constraints.
[2]Skip mantissa multiplication under hardware constraints.

Fig. 8. Operation of the RSAB-IMA.

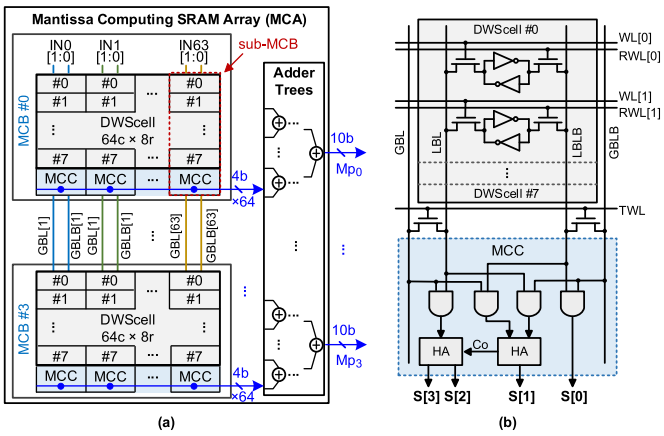


Fig. 9. (a) Structure of the proposed MCA. (b) Structure of sub-MCB.

mantissas can be directly accumulated. It needs to be stated that targeting the hardware optimization method proposed in Section II-B, we put forward the following hardware constraint conditions: for Esum > Emax, INM performs no shift; for Esum < Emax-7, mantissa multiplication is bypassed.

B. Proposed MCA Scheme

Fig. 9(a) illustrates the structure of the proposed MCA, which consists of four MCBs and four adder trees. Each MCB integrates 64 sub-MCBs that share common horizontal control signals (WL/RWL), while all four MCBs are interconnected through global BL pairs (GBL/GBLB). In this scheme, the 8-bit WM[7:0] is divided into four segments and, respectively, stored in the four MCBs. During computation, the horizontal control signals of the same row of DWScell in the four MCBs are activated simultaneously, and the 2-bit WM is transmitted to the MCC via the local BLs (LBLs/LBLBs). Each MCC executes precise multiplication between the incoming 2-bit WM and the 2-bit input mantissa. This distributed computation scheme achieves high throughput while minimizing interconnect overhead.

Fig. 9(b) shows the structure of the sub-MCB, which consists of eight DWScells and one MCC. Inspired by [31], each DWScell consists of dual wordline-separated 6T SRAM cells, allowing 2-bit WM to be processed within a single cycle. Compared with single-bit schemes, this configuration theoretically doubles mantissa computation throughput under identical cycle duration and array size. According to the

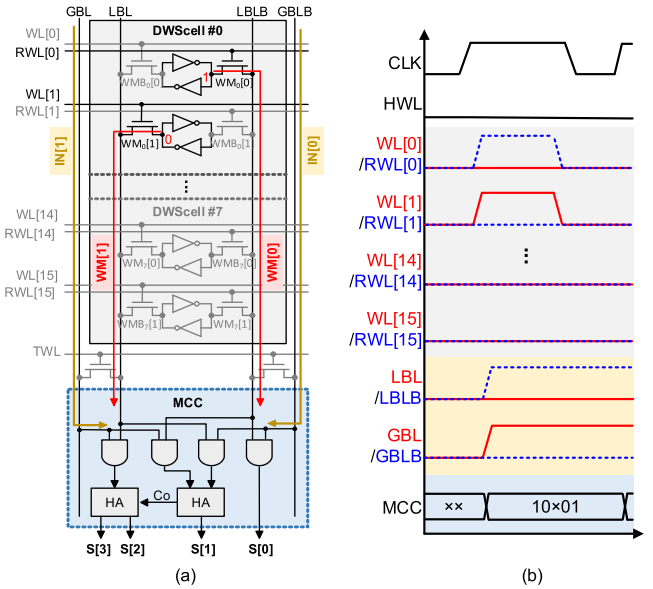


Fig. 10. (a) Operation of sub-MCB in calculation mode. (b) Waveform of sub-MCB in calculation mode.

multiplication calculation rules, the MCC consists of four ANDs and two HAs.

The sub-MCB has two operating modes.

1) Traditional SRAM mode that supports write operation.
2) Calculation mode that supports 2-bit multiplication (read operation is also executable in this mode).

Its write operation follows a scheme similar to that of a conventional wordline-decoupled 6T cell. The operation and corresponding waveform of the sub-MCB in computation mode are depicted in Fig. 10(a) and (b), respectively.

During the calculation phase, TWL is held at a low level, while the GBL/GBLB and LBL/LBLB are electrically isolated. The mantissa input data, supplied via GBL/GBLB, and the weight data, read from the memory cells onto LBL/LBLB, are processed within the MCC. As an illustrative example, at a specific time instance, the WL and RWL of DWScell#0 in MCB#0 are activated, whereas DWScell#1–7 remain inactive. In this configuration, weight bits WM0 [1] and WM0[0] are read onto LBL and LBLB, respectively, while the input mantissa bits IN [1] and IN[0] are applied to GBL and GBLB. At this stage, the MCC carries out a 2-bit multiplication operation.

For four sub-MCBs that share the same GBL/GBLB, assuming that at clock cycle T, the GBL[j] and GBLB[j] are assigned the input data IN [1] and IN[0], respectively. Then, the four MCCs will, respectively, calculate IN[1:0] × WM[1:0], IN[1:0] × WM[3:2], IN[1:0] × WM[5:4], and IN[1:0] × WM[7:6]. After completing the multiplication in MCC, all the 4-bit results of MCB#0–MCB#3 are summed by four adder trees.

In this study, we also proposed a novel 5T HA and an AND logic in MCC, with their circuit schematics illustrated in Fig. 11. Compared with the conventional HA, which requires at least 16 MOS transistors, this 5T HA achieves a 69% reduction in transistor count. Similar to the 9T FA, output

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

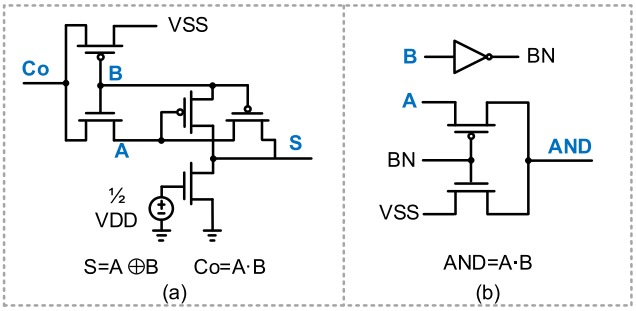


Fig. 11. (a) Proposed 5T HA. (b) Proposed AND logic.

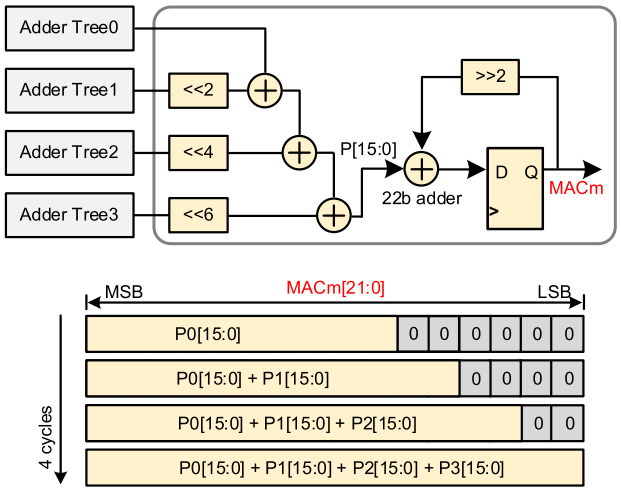


Fig. 12. Accumulation process of the accumulator within four calculation cycles.

buffers are also used at the outputs of the proposed 5T HA to achieve a full-swing output.

C. Mantissa Accumulation Details

To finish the 8-bit input mantissa multiplication, four computation cycles are required. Therefore, an extra accumulator must be introduced to finish the whole calculation, as shown in Fig. 12.

During the first cycle, the outputs of four adder trees are accumulated by three adders to generate P0[15:0], which forms the upper 16 bits of the 22-bit adder's input, while zeros are fed from the REG output as the second input. During the second cycle, the newly computed P1[15:0] is added with the 2-bit right-shifted data stored in REG through the 22-bit adder. The third and fourth cycles replicate the procedure of the second cycle.

V. Simulation and Performance Analysis

This work uses a 28-nm CMOS logic process to simulate the proposed FP-CIM macro, which consists of two SRAM array blocks: an exponent array and a mantissa array, each with a size of 4 kb. This section analyzes the simulation results for the performance of this macro and compares it with previous works.

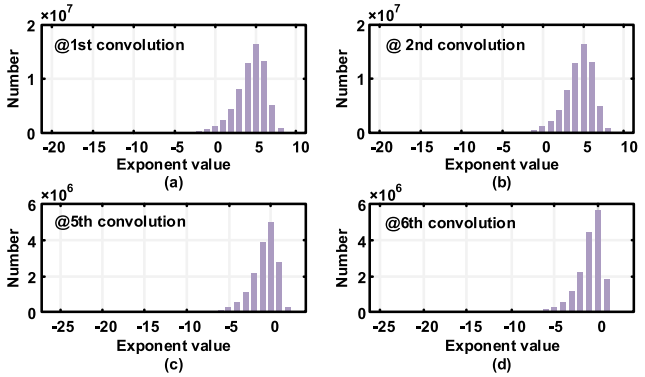


Fig. 13. Exponent distribution of the output after (a) first, (b) second, (c) fifth, and (d) sixth convolutions performed by VGG16 during an inference task.

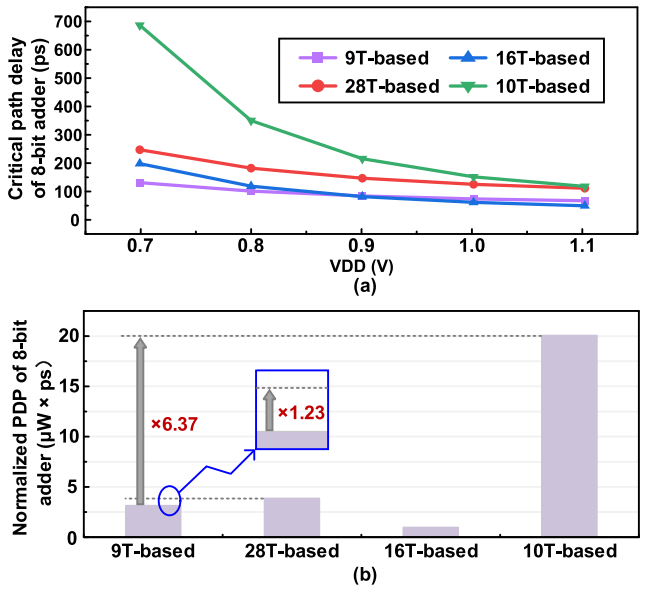


Fig. 14. (a) Critical path delay of four types of 8-bit adders. (b) Normalized PDP of four types of 8-bit adders.

Fig. 13 shows the exponent distribution of the outputs from the first to second and fifth to sixth convolutions of the VGG16 network during an inference task. For most convolutional layers, the exponent distribution of their output results is concentrated within a certain range, exhibiting a long-tail distribution characteristic. Therefore, the impact of appropriately configuring the maximum exponent outside the chip on inference accuracy is minimal. Compared with the traditional FPMAC scheme, this scheme eliminates the area and power consumption overhead associated with the maximum exponent detection circuitry, significantly enhancing system performance.

Fig. 14 presents the critical path delay and normalized PDP of four types of 8-bit adders implemented with the proposed 9T FA and the existing 28T, 16T, and 10T FAs, respectively. As shown in Fig. 14(a), compared with the 28T-based and 10T-based 8-bit adders, the 8-bit adder based on the proposed 9T FA achieves a smaller maximum delay. When compared with the 16T-based 8-bit adder, the 9T-based design exhibits

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

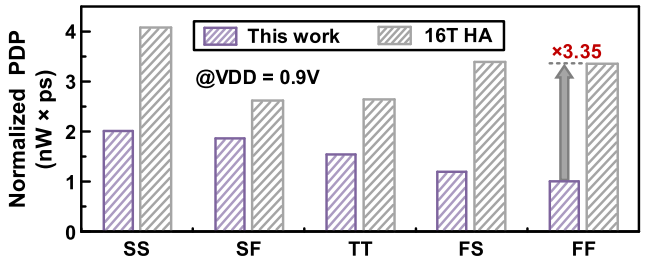


Fig. 15. Normalized power–delay product of the proposed 5T HA and the traditional 16T HA at different process corners.

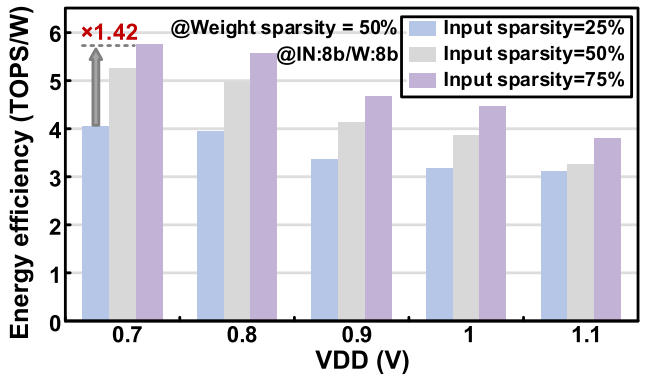


Fig. 16. Energy efficiency of mantissa MAC operations of different voltages.

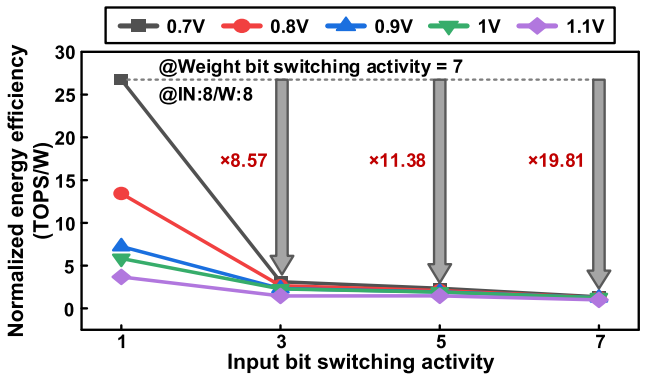


Fig. 17. Normalized energy efficiency of mantissa MAC operations for different input BSAs.

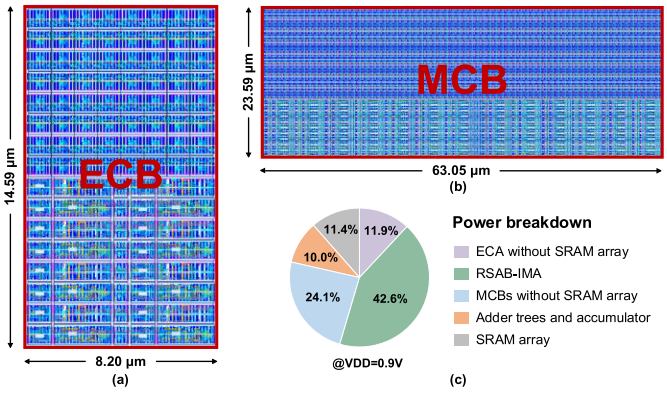


Fig. 18. (a) Layout of the proposed ECB. (b) Layout of the proposed MCB. (c) Power breakdown of the proposed FP-CIM macro.

a greater advantage at lower VDD values. From Fig. 14(b), it can be observed that the 9T-based 8-bit adder achieves a lower PDP than the 28T-based and 10T-based adders, indicating better overall performance in terms of both power consumption and speed. Compared with the 16T-based adder, the advantage of the 9T-based design lies in its smaller area overhead, which is crucial for large-scale integrated adder arrays. These results collectively demonstrate the superior performance of the proposed 9T FA in terms of speed, energy efficiency, and area efficiency.

Fig. 15 shows the normalized PDP of the 5T HA and the traditional 16T HA under five different process corners (@0.9 V). The results show that the 5T HA achieves a 3.35× lower PDP than the 16T HA, with this improvement being most significant at the FF corner.

Figs. 16 and 17 illustrate the EF of mantissa MAC operations under different supply voltages, input sparsities, and input bit switching activities (BSAs). Sparsity is defined as the proportion of zeros in the data, while BSA is the number of transitions between adjacent bits. For example, the BSA of 00001111 is 1, and that of 00110011 is 3. The BSA reflects the degree of dispersion of 1 and 0 in the data.

In the simulation shown in Fig. 16, the weight sparsity was fixed at 50%, the weight BSA was fixed at 7, and the input BSA was fixed at 4. As evidenced by the data, the mantissa MAC EF exhibits consistent improvement at higher input sparsity levels. Under conditions where VDD was 0.7 V and the input sparsity was 75%, the EF is 1.42× compared with a 25% input sparsity.

In the simulation shown in Fig. 17, the input sparsity was fixed at 50%, and the weight sparsity was fixed at 0%. The results demonstrate that in most cases, when the BSA is fixed, the EF remains almost unchanged with variations in the supply voltage. However, at 0.7-V VDD, EF achieves 19.81× the value at an input BSA of 1 compared with that at an input BSA of 7. As shown, we conclude that mantissa computation EF in our design is strongly sensitive to data dispersion. This phenomenon is primarily attributed to the block-store capability inherent in the MCA.

As shown in Fig. 18(a) and (b), our proposed ECB and MCB layouts have areas of 119.64 and 1487.35 μm^2, respectively. Fig. 18(c) shows the power breakdown of the proposed macro when both weight and input sparsity are 50%. It can be seen that the RSA-based input mantissa alignment unit accounts for 42.6% of the total power consumption, while the SRAM array accounts for 11.4%.

Fig. 19 shows the EF and throughput of this macro at different supply voltages. At a supply voltage of 0.7 V, the macro achieves an EF of 4.81 TFLOPS/W, and at 1.1 V, its throughput reaches 16.3 GFLOPS.

Table I summarizes a comparison of the performance of this work with recent CIM works. A comparative analysis shows that our macro achieves higher energy efficiency than current state-of-the-art design [29], [33], [35]. In comparison to [28], the architecture achieves higher throughput due to its superior

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON TO PRIOR CIM WORKS

	ISSCC'22[28]	ESSCIRC'22[33]	ISCAS'23[29]	JSSC'24[31]	L-SSC'25[34]	TCAS-I'25[35]	This work
Technology	28nm	28nm	40nm	28nm	28nm	28nm	**28nm**
Array size	192Kb	16Kb	N/A	64Kb	4Kb	N/A	**8Kb**
Cell structure	N/A	8T/10T	6T	DBcell	9T	ReRAM	**6T/DWScell**
Domain	Digital	Digital	Digital	Digital	Analog	Analog	**Digital**
Input precision (bit)	INT8, INT16/ BF16, FP32	FP8-FP32	BF16/FP32	INT8/BF16	FP16	INT8/FP16	**BF16**
Weight precision (bit)	INT8, INT16/ BF16, FP32	FP8-FP32	BF16/FP32	INT8/BF16	FP16	INT8/FP16	**BF16**
Model	CNN	DNN	N/A	CNN	CNN	DNN	**CNN**
Supply voltage (V)	0.6-1.0	0.5-0.9	0.7-1.1	0.6-0.9	0.6-1	0.8	**0.7-1.1**
Throughput (*GOPS or GFLOPS)	*1.35/1.08(1.0V)	N/A	56.5/16(1.1V)	*112.3-375/ 89.8-300	0.82-3.27	N/A	**9.19-16.3**
Energy efficiency (*TOPS/W or TFLOPS/W)	36.5(INT8)/ 29.2(BF16) (0.65V)	1.644-0.099 (0.5V)	1.97/0.62 (0.7V)	*44/31.6(0.6V)	6.04(0.1GHz)	*2.43/1.24	**4.81(0.7V)**
Inference accuracy (%) (CIFAR100)	N/A	N/A	N/A	[1]75.9	N/A	[4]74.6	**76.7**
mAP (%) (COCO)	N/A	N/A	N/A	[2]52.4	N/A	N/A	**50.2**
MIoU (%) (VOC2012)	N/A	N/A	N/A	[3]56.5	N/A	N/A	**59.1**

[1] The neural network model used is ResNet-18. [2] The neural network model used is Yolov3. [3] The neural network model used is U-net.
[4] The neural network model used is VGG-13.

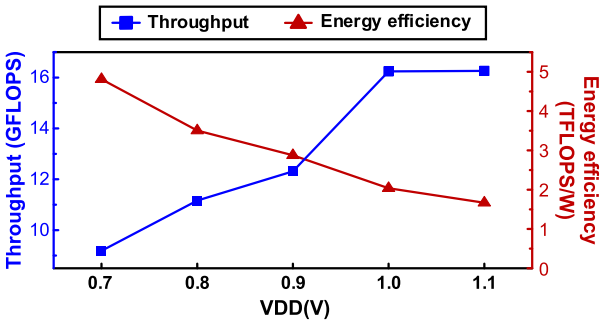


Fig. 19. Measured energy efficiency and throughput of different voltages.

computation speed. Based on the Emax configuration scheme, our design achieves 76.7% accuracy, 50.2% mean average precision (mAP), and 59.1% mean intersection over union (MIoU) on ResNet50 (@CIFAR-100), Yolov5 (@COCO), and U-Net (@VOC2012) models, respectively.

VI. CONCLUSION

This work proposes an all-digital SRAM-based FP-CIM macro for CNNs, which achieves a balanced optimization among area, EF, and computational accuracy. The codesign of the Emax dynamic configuration scheme based on the long-tail distribution and the customized FA effectively simplifies the exponent processing flow, enables the reuse of exponent addition and subtraction circuits, and enhances energy efficiency. The proposed MCB scheme composed of DWScell and MCC, coupled with the customized HA, achieves error-free 2-bit mantissa multiplication, thereby improving both throughput and area efficiency while maintaining computational accuracy. The simulation results demonstrate that the macro achieves 4.81 TFLOPS/W energy efficiency during BF16 MAC operations at 0.7-V supply.

REFERENCES

[1] Z. Jiang, S. Yin, J.-S. Seo, and M. Seok, "C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020, doi: 10.1109/JSSC.2020.2992886.

[2] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020, doi: 10.1109/JSSC.2019.2963616.

[3] J. Wang et al., "14.2 a compute SRAM with bit-serial integer/floating-point operations for programmable in-memory vector acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2019, pp. 224–226, doi: 10.1109/ISSCC.2019.8662419.

[4] Q. Dong et al., "A 0.3 V VDDmin 4+2T SRAM for searching and in-memory computing using 55nm DDC technology," in *Proc. Symp. VLSI Circuits*, Kyoto, Japan, Jun. 2017, pp. C160–C161, doi: 10.23919/VLSIC.2017.8008465.

[5] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and trends of SRAM-based computing-in-memory for AI edge devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 1773–1786, May 2021, doi: 10.1109/TCSI.2021.3064189.

[6] X. Si et al., "A local computing cell and 6T SRAM-based computing-in-memory macro with 8-b MAC operation for edge AI chips," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, Sep. 2021, doi: 10.1109/JSSC.2021.3073254.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.

[7] X. Si et al., "A twin-8T SRAM computation-in-memory unit-macro for multibit CNN-based AI edge processors," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 189–202, Jan. 2020, doi: 10.1109/JSSC.2019.2952773.

[8] J.-W. Su et al., "Two-way transpose multibit 6T SRAM computing-in-memory macro for inference-training AI edge chips," *IEEE J. Solid-State Circuits*, vol. 57, no. 2, pp. 609–624, Feb. 2022, doi: 10.1109/JSSC.2021.3108344.

[9] Y. Zhang et al., "Single-mode CMOS 6T-SRAM macros with keeper-loading-free peripherals and row-separate dynamic body bias achieving 2.53fW/bit leakage for AIoT sensing platforms," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 184–186, doi: 10.1109/ISSCC42614.2022.9731573.

[10] P.-C. Wu et al., "A 28nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3, doi: 10.1109/ISSCC42614.2022.9731681.

[11] C. Yu, T. Yoo, T. T.-H. Kim, K. C. Tshun Chuan, and B. Kim, "A 16K current-based 8T SRAM compute-in-memory macro with decoupled read/write and 1-5bit column ADC," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Boston, MA, USA, Mar. 2020, pp. 1–4, doi: 10.1109/CICC48029.2020.9075883.

[12] S. Yue et al., "CV-CIM: A hybrid domain Xor-derived similarity-aware computation-in-memory supporting cost-volume construction," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 719–733, Feb. 2025, doi: 10.1109/JSSC.2024.3421589.

[13] S. Um, S. Kim, S. Hong, S. Kim, and H.-J. Yoo, "LOG-CIM: A 116.4 TOPS/W digital computing-in-memory processor supporting a wide range of logarithmic quantization with zero-aware 6T dual-WL cell," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Haikou, China, Nov. 2023, pp. 1–3, doi: 10.1109/A-SSCC58667.2023.10347959.

[14] P. Chen, M. Wu, W. Zhao, Y. Ma, T. Jia, and L. Ye, "A 44.3 TOPS/W SRAM compute-in-memory with near-CIM analog memory and activation for DAC/ADC-less operations," *IEEE Solid-State Circuits Lett.*, vol. 7, pp. 299–302, 2024, doi: 10.1109/LSSC.2024.3418099.

[15] Y. Fu et al., "FLEX-CIM: A flexible kernel size 1-GHz 181.6-TOPS/W 25.63-TOPS/mm^2 analog compute-in-memory macro," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3021–3031, Sep. 2024, doi: 10.1109/JSSC.2024.3386192.

[16] P.-C. Wu et al., "A 22nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2-70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 126–128, doi: 10.1109/ISSCC42615.2023.10067527.

[17] A. Guo et al., "A 28 nm 128-kb exponent- and mantissa-computation-in-memory dual-macro for floating-point and INT CNNs," *IEEE J. Solid-State Circuits*, vol. 60, no. 10, pp. 3639–3654, Oct. 2025, doi: 10.1109/JSSC.2025.3558322.

[18] K. Yoshioka, "A 818–4094 TOPS/W capacitor-reconfigured analog CIM for unified acceleration of CNNs and transformers," *IEEE J. Solid-State Circuits*, vol. 60, no. 5, pp. 1844–1855, May 2025, doi: 10.1109/JSSC.2024.3457898.

[19] J.-W. Su et al., "15.2 a 28nm 64Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 240–242, doi: 10.1109/ISSCC19947.2020.9062949.

[20] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, "A programmable heterogeneous microprocessor based on bit-scalable in-memory computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 9, pp. 2609–2621, Sep. 2020, doi: 10.1109/JSSC.2020.2987714.

[21] Y.-C. Chiu et al., "A 4-Kb 1-to-8-bit configurable 6T SRAM-based computation-in-memory unit-macro for CNN-based AI edge processors," *IEEE J. Solid-State Circuits*, vol. 55, no. 10, pp. 2790–2801, Oct. 2020, doi: 10.1109/JSSC.2020.3005754.

[22] J. Yue et al., "15.2 a 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 238–240, doi: 10.1109/ISSCC42613.2021.9365958.

[23] J. Yue et al., "STICKER-IM: A 65 nm computing-in-memory NN processor using block-wise sparsity optimization and inter/intra-macro data reuse," *IEEE J. Solid-State Circuits*, vol. 57, no. 8, pp. 2560–2573, Aug. 2022, doi: 10.1109/JSSC.2022.3148273.

[24] B. Yan et al., "A 1.041-Mb/mm^2 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 188–190, doi: 10.1109/ISSCC42614.2022.9731545.

[25] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3, doi: 10.1109/ISSCC42614.2022.9731754.

[26] V. Sharma, X. Zhang, N. S. Dhakad, and T. T.-H. Kim, "FlexDCIM: A 400 MHz 249.1 TOPS/W 64 kb flexible digital compute-in-memory SRAM macro for CNN acceleration," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 10, pp. 5696–5707, Oct. 2025, doi: 10.1109/TCSI.2025.3547853.

[27] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.

[28] F. Tu et al., "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3, doi: 10.1109/ISSCC42614.2022.9731762.

[29] Y. Mai, M. Wang, C. Zhang, B. Zhong, and Z. Yu, "A 1.97 TFLOPS/W configurable SRAM-based floating-point computation-in-memory macro for energy-efficient AI chips," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Monterey, CA, USA, May 2023, pp. 1–5, doi: 10.1109/ISCAS46773.2023.10182197.

[30] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "FP-IMC: A 28 nm all-digital configurable floating-point in-memory computing macro," in *Proc. ESSCIRC IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Lisbon, Portugal, Sep. 2023, pp. 405–408, doi: 10.1109/ESSCIRC59616.2023.10268770.

[31] A. Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep. 2024, doi: 10.1109/JSSC.2024.3375359.

[32] J. Yue et al., "A 28nm 16.9-300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 1–3, doi: 10.1109/ISSCC42615.2023.10067779.

[33] S. Jeong, J. Park, and D. Jeon, "A 28nm 1.644TFLOPS/W floating-point computation SRAM macro with variable precision for deep neural network inference and training," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2022, pp. 145–148, doi: 10.1109/ESSCIRC55480.2022.9911450.

[34] W.-C. Wang, S. Zhang, L. Shamieh, N. V. Kidambi, I. Chakraborty, and S. Mukhopadhyay, "MIX-ACIM: A 28-nm mixed-precision analog compute-in-memory with digital feature restoration for vector-matrix multiplication," *IEEE Solid-State Circuits Lett.*, vol. 8, pp. 213–216, 2025, doi: 10.1109/LSSC.2025.3590757.

[35] Z. Zhou et al., "ArPCIM: An arbitrary-precision analog computing-in-memory accelerator with unified INT/FP arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2229–2242, May 2025, doi: 10.1109/TCSI.2024.3491825.



Wenjuan Lu (Member, IEEE) received the M.S. degree in circuits and systems and the Ph.D. degree in microelectronics and solid-state electronics from Anhui University, Hefei, China, in 2014 and 2017, respectively.

She is an Associate Professor of Microelectronics and Solid-State Electronics with Anhui University. Her current research interests include next-generation nonvolatile memory and the circuit design of low-power, high-performance, and high-reliability memory.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.



Can Zhang received the B.S. degree in electronic information science and technology from Henan University, Kaifeng, China, in 2024. She is currently working toward the M.S. degree in integrated circuit science and engineering at Anhui University, Hefei, China.

Her current research interests include static random access memory and in-memory computing circuits.



Wei Hu received the M.S. degree in integrated circuit engineering from Anhui University, Hefei, China, in 2025.

Her research interests include static random access memory and floating-point in-memory computing circuits.



Xiaobo Gong received the B.S. degree in photonic information science and engineering from Yangtze University, Jingzhou, China, in 2021. He is currently working toward the M.S. degree in integrated circuit engineering at Anhui University, Hefei, China.

His research interests mainly focus on static random access memory and computing-in-memory circuits.



Zhihao Li received the B.S. degree in polymer material and engineering from Qingdao University, Qingdao, China, in 2022. He is currently working toward the M.S. degree in integrated circuit engineering at Anhui University, Hefei, China.

His research interests mainly focus on static random access memory, nonvolatile memory, and computing-in-memory circuits.



Shijie Xu recevied the B.S. degree in microelectronics science and engineering from Henan Normal University, Xinxiang, China, in 2023. He is currently working toward the M.S. degree in microelectronics and solid-state electronics at Anhui University, Hefei, China.

His current research interests include static random access memory and computing-in-memory.



Xiulong Wu (Senior Member, IEEE) received the B.S. degree in computer science from the University of Science and Technology of China (USTC), Hefei, China, in 2001, and the M.S. and Ph.D. degrees in electronic engineering from Anhui University, Hefei, in 2005 and 2008, respectively.

From 2013 to 2014, he was a Visiting Scholar with the Engineering Department, The University of Texas at Dallas, Richardson, TX, USA. He is currently a Professor at Anhui University. He has published about 60 articles and holds over ten Chinese patents. His research interests include high-performance static random access memory and mixed-signal ICs. His current research interests include low-power circuit design, computing-in-memory, and nonvolatile memory and system design.



Chunyu Peng (Senior Member, IEEE) received the M.S. degree in circuits and systems and the Ph.D. degree in microelectronics and solid-state electronics from Anhui University, Hefei, China, in 2013 and 2022, respectively.

He is currently an Associate Professor of Microelectronics and Solid-State Electronics with Anhui University. His research interests include signal processing, analog IC design, and high-performance, reliable memory technology.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:36:48 UTC from IEEE Xplore. Restrictions apply.